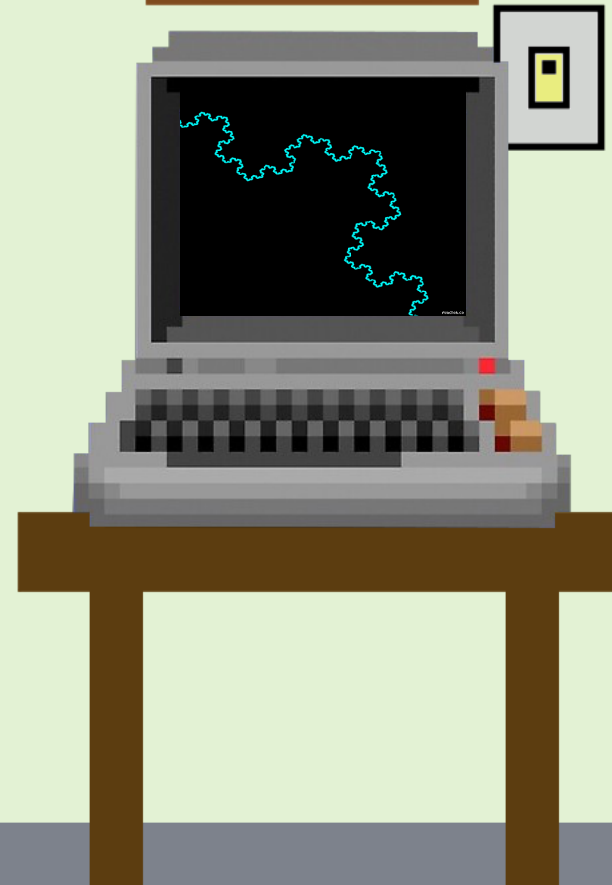


Haskell

Programação Funcional



Haskell

- Linguagem de programação puramente funcional
- Elaborada em 1987 e batizada em homenagem ao matemático Haskell Curry
- Sua estrutura primária são as funções
- Muito utilizada no meio acadêmico
- Derivada de outras linguagens funcionais
- Possui uma variedade de aplicações
- Apesar de simples é muito poderosa



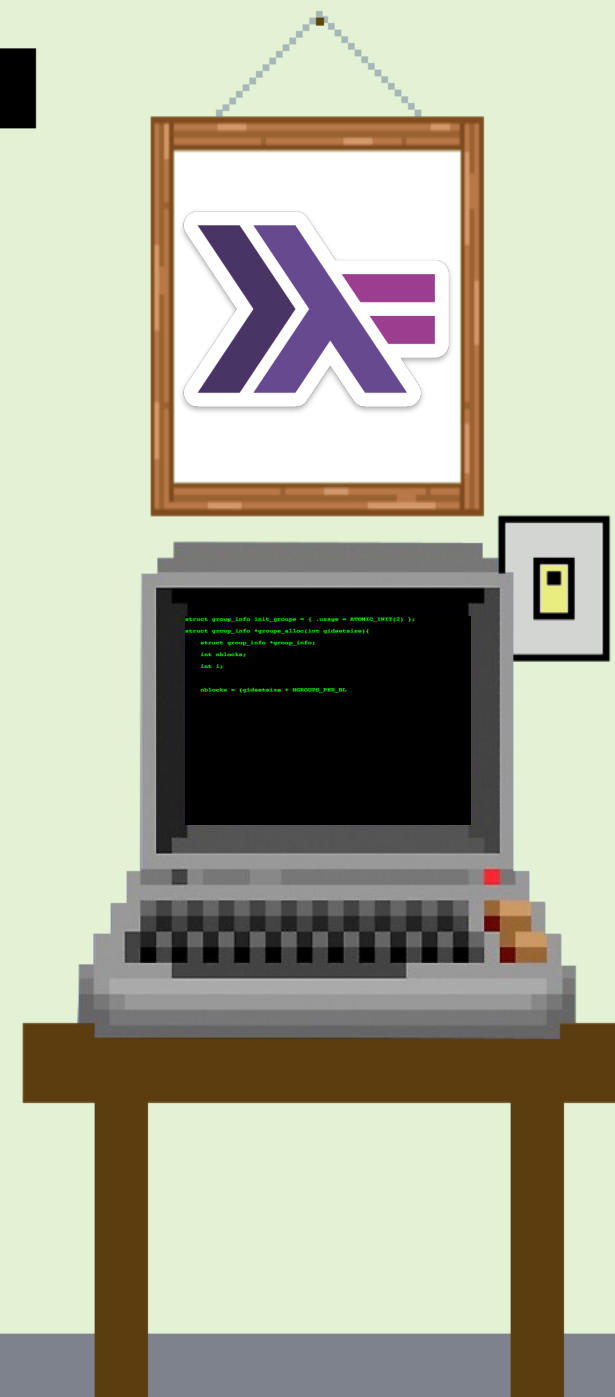
Haskell

- Antes de Haskell existiam mais de uma dúzia de linguagens funcionais
- A falta de uma linguagem funcional comum impedia a disseminação do uso deste paradigma
- Foi criado um comitê para projetar a criação de uma nova linguagem funcional



Projeto - Haskell

- Ser adequada ao ensino e indústria
- Ser completamente descrita através da publicação de uma sintaxe e semântica formal
- Disponibilizada livremente
- Baseada em ideias que tenham amplo consenso
- Reduzir a diversidade desnecessária de linguagens de programação funcionais



Recursos

- Funções de alta-ordem
- Tipagem polimórfica
- Tipos algébricos de dados definidos pelo usuário
- Casamento de padrões
- Compreensão de listas
- Sistema modular
- Sistema de E/S monádico
- Rico conjunto de tipos de dados primitivos



Desenvolvimento

- Arquivos .hs
 - Padrão
 - Todo o texto é programa, exceto comentários
 - comentário de linha "--"
 - {- comentário varias linhas -}
- Arquivo .lhs
 - Literate mode
 - Todo o texto é considerado comentário



Exemplo de Código

```
{- #####  
exemplo.hs  
Este arquivo eh um exemplo de um arquivo .hs. Deve ser editado como arquivo  
texto e salvo com a extensao .hs.  
##### -}  
valor :: Int                                -- Uma variavel inteira  
valor = 39  
  
novalinha :: Char                           -- 0 caractere de nova linha  
novalinha = '\n'  
  
resposta :: Bool                            -- Uma variavel booleana  
resposta = True  
  
maior :: Bool                               -- Uma variavel booleana  
maior = (valor > 71)  
  
quadrado :: Int -> Int                      -- Uma funcao de Z em Z  
quadrado x = x*x  
  
todosIguais :: Int -> Int -> Int -> Bool -- Uma funcao de ZxZxZ em Bool  
todosIguais n m p = (n == m) && (m == p)  
{- ##### -}
```



Desenvolvimento

- Um programa é construído utilizando módulos
- Módulos consistem em uma coleção de declarações
- Uma declaração pode ser feita para classes de tipos, valores ordinários, tipos e informações de fixação
- O nível mais baixo de um programa Haskell é formado pelas expressões
 - Denotam um valor e possuem tipo estático



Desenvolvimento

- Interpretador interativo
 - Avalia expressões, explora módulos e testa código
- Pode ser inicializado através do comando ghci

Comando	Ação realizada
:?	Aciona o help
:e	Chama o script atual
:e exemplo.hs	Edita o arquivo exemplo.hs
:l exemplo.hs	Carrega o script exemplo.hs e limpa outros arquivos carregados
:a exemplo.hs	Carrega o script exemplo.hs sem limpar os outros arquivos
:q	Termina a seção



Expressões

- Em geral, funções são declaradas como pré-fixas (forma mais utilizada na maioria das linguagens)
 - Ex: `succ 8`, `min 9 10`, `max 100 102`
- Operadores e funções atuam sobre tipos específicos
 - Ex: `True && 1`, `succ "ds"`, `min 9 'a'`



Identificadores

- Identificadores são case-sensitive
 - Tipos começam sempre com letra maiúscula

```
type Par = (Int, Int)
```

- Definições (constantes), variáveis e funções começam sempre com letra minúscula

```
somaAmbos :: Par -> Int
```

```
somaAmbos (primeiro, segundo) = primeiro + segundo
```

- A primeira letra pode ser seguida por outros caracteres (letras, dígitos, sublinhados, etc.)



Identificadores

- Palavras reservadas da linguagem são sempre escritas em letras minúsculas

case	else	infix	module	type
class	hiding	infixl	of	where
data	if	infixr	renaming	
default	import	instance	then	
deriving	in	let	to	



```
data GroupInfo (info :: Group) = { group :: GROUP_ID, info :: () }
data GroupInfo (info :: Group) = { group :: GROUP_ID, info :: () }
data GroupInfo (info :: Group) = { group :: GROUP_ID, info :: () }
data GroupInfo (info :: Group) = { group :: GROUP_ID, info :: () }
data GroupInfo (info :: Group) = { group :: GROUP_ID, info :: () }
```

Atividade

- Leitura do capítulos 3
- Responder os exercícios

